

DNDROM / RESEARCH

Believable worlds without boundaries

Local AI · Procedural environments · Infinite-world rendering

Deep research and implementation direction | 5 September 2026

Audience: DnDRom product and engineering. Scope: local AI, procedural fantasy environments, enterable settlements, indefinite world expansion, distant views, and the current Tauri / PlayCanvas / WebGPU implementation. Recommendations assume a desktop target and an adjustable quality budget; no minimum GPU or frame-rate guarantee has yet been established.

A practical visual direction

Mountains and watersheds. Forests and fields. Streets and enterable buildings. Materials, weather and distant views. One continuous world revealed through outward-moving volumetric fog.

Evidence-backed recommendations and source audit. Performance targets and proposed features are clearly separated from validated implementation.

The decision

DnDRom needs a connected environment generator and a hierarchy of representations. Adding more noise, primitives, or independently generated textures will not reliably produce the requested visual quality. The recommended pipeline is: world geography and story intent; watersheds and climate; roads and land use; plant communities and architectural programs; material and weather response; then geometry and shading appropriate to the current view.

The local model should author relationships and reusable rules: where a village belongs, what its construction language is, what grows nearby, and how references change its appearance. Procedural systems should solve, validate, generate, and render those rules. Seeds preserve the chosen result across travel, saves and detail levels. Repeatability does not limit the model to a menu of predetermined scenes.

This is an architectural recommendation derived from the research, not a claim that the current application already implements it. The most useful production precedent is rule-driven generation around the player: Guerrilla describes a GPU placement pipeline controlled by artists, while Hello Games reports moving planetary object rendering to a GPU-based system and improving texture streaming and LOD generation. Neither establishes a transferable frame-rate result for our app. [Guerrilla, Jaap van Muijden, GPU-Based Procedural Placement, 2017](#). [Hello Games, Worlds Part I, 2024](#).

Highest-impact changes: establish shared world coordinates and distant terrain; fix landform scale and drainage; replace rounded foliage masses with botanical silhouettes and layered forests; make roads, plots, building entrances and interiors agree; unify materials, sun, atmosphere and weather. Evaluate these changes together in complete scenes.

Success means: a forest feels enclosed at eye level and coherent from above; fields occupy plausible land and have readable boundaries; mountains have ridges, valleys and exposed rock; a distant settlement retains its identity when entered; rain changes surfaces according to exposure; references alter structure as well as palette; performance remains bounded during movement.

1. What the current implementation reveals

These are source observations from this workspace, not deductions from a technology demo. The two saved WebGPU screenshots inspected are dated 4 September and predate the newest working changes. They show a finite board, broad rounded water areas, strongly repeated vegetation, conspicuously dark bridge/roof surfaces, and a road with little settlement context. They establish an earlier visual baseline, not the appearance of every current scene.

Terrain and rivers. In `worldProcedural.ts`, mountains largely come from a biome relief multiplier over several noise bands. Regional elevation is added, but landform shape still depends on the base biome. The coupled particle routine rounds movement to neighboring grid cells and deposits only part of its remaining sediment at termination. These are important departures to investigate before calling it a faithful implementation of the supplied river method.

Roads. The current search has eight directions and no heading in its state. Its output is simplified and smoothed, then the compiler smooths the road again for display. This can make the painted route disagree with the route used to grade terrain and clear vegetation.

Vegetation. Branch generation already uses space colonization. Crowns still use large ellipsoids; broad botanical styles and a handful of variants provide limited species and age diversity. Moisture/slope/flow masks exist, but there is no complete temperature, rainfall, season and plant-community model.

Materials and sky. `tabletopShaders.ts` starts its altitude-based snow blend around 5–10.5 world units. `volumetricClouds.ts` supplies a fixed sun vector independently of the lighting rig. Terrain selects a dominant biome per chunk. These rules can create low-altitude snow, contradictory illumination and visible biome boundaries.

Scale and cost. The earlier travel fixture is 84.6 MB of JSON: approximately 32.6 MB water entities, 26.6 MB grass and 16.9 MB repeated tree descriptions. It contains 208,250 grass clusters and 589 trees. The failed native travel run reached roughly 11 GB renderer memory; its root cause has not been isolated sufficiently to attribute all of that growth to one subsystem. Current grass compute culls whole batches, not every blade or plant.

Work in progress. The latest edits introduce seed-based tree batches/shared meshes, align geometry creation with resident LOD, release grass instance buffers, cache the chunk quadtree, and reduce water sampling/detail. Typechecking and 46 focused domain tests pass. These are not evidence of a visually approved release, infinite traversal, or a resolved native memory problem.

2. An indefinitely expanding world and a stable horizon

Geometry clipmaps keep nested terrain grids around the viewer, update only newly exposed areas, and morph boundaries between resolutions. Their workload depends on retained levels and grid resolution rather than the total world area. This directly fits the long-distance view requirement. The published historical performance numbers are irrelevant as a promise for our workload. [Asirvatham and Hoppe, GPU Gems 2, Terrain Rendering Using GPU-Based Geometry Clipmaps, 2005](#).

Recommended architecture. Use a sparse world tile hierarchy for storage/generation and nested clipmap grids for the terrain view. These serve different purposes and can coexist. Derive all detail levels from the same geographic description plus cached erosion and edit residuals. Coarse terrain must retain the important summit, valley and shoreline silhouettes that local terrain later refines.

Store a persistent integer tile address and a local metric offset. Render positions relative to an origin near the camera; rebase terrain, objects, particles, physics and camera history together. A second world pass or a verified depth-precision strategy should handle the very distant range. Increasing `farClip` alone is insufficient. Epic's large-world documentation demonstrates translated world space and higher-precision representations; these are concepts to adapt, not an instruction to import Unreal internals. [Epic Games, Large World Coordinates Rendering, current documentation accessed September 2026](#).

Initial distance bands for testing, **not fixed production limits**:

View range	Representation	Persistent authority
Near, roughly 0–100 m	Detailed terrain, leaves, doors, props, navigation	World tile, recipes, edits, room graph
Middle, roughly 100 m–2 km	Simplified terrain/buildings; clustered or image-based trees	Same locations and prototypes
Far, roughly 2–30 km	Coarse terrain, canopy coverage, settlement roof/wall/tower silhouettes	Same geographic hierarchy
Beyond, when visible	Additional coarse levels plus atmospheric attenuation	Same seed and geography; bounded cache

Select LOD by projected error, camera elevation, silhouette importance and budget, with hysteresis. Distance bands initialize tuning; they should not make a large nearby cliff as cheap as a flat field at the same range. Preserve parent tiles until children and shared edges are ready. Cancel stale jobs, prioritize the view and likely travel direction, and cap generation/upload work per frame.

The world can extend indefinitely while each frame has a finite visibility and memory budget. Distant Horizons is a useful perceptual reference, but a horizon should contain lower-detail versions of places that can actually be visited. Remove the existing fixed 256-unit fit and compressed settlement-distance presentation from geographic authority. Travel overview and local play should share coordinates; change camera and representation, not location identity.

3. Mountains, rivers and fields must have a shared structure

The supplied McDonald article improves droplet motion, sediment behavior and persistent discharge/momentum coupling to produce meandering channels. It is valuable for erosion and river development, but it is not an infinite-world scheduler, a climate model or a complete mountain generator. Its finite simulations do not by themselves solve drainage consistency between independently generated tiles. [Nick McDonald, Procedural Hydrology: Improvements and Meandering Rivers, 12 December 2023.](#)

For broad geography, hydrology-led terrain modeling is a complementary approach. G  nevaux and colleagues generate terrain from river networks and a hierarchical continuous representation, with mountains, hills and river features contributing to the result. [G  nevaux et al., Terrain Generation Using Procedural Models Based on Hydrology, ACM TOG, July 2013.](#)

Recommended synthesis: define mountain ranges, major ridges, outlets and drainage basins at regional scale; generate subordinate valleys and tributaries; run bounded erosion over cached regions with explicit boundary conditions; derive soils, slopes, sediment and surface water; then place ecosystems and infrastructure. Detail noise belongs after the large forms are readable. A mountain should have connected ridgelines, relief over a meaningful horizontal extent, ravines, talus and a transition to foothills.

Use continuous particle positions, bilinear sampling and deposition, controlled time steps, conserved sediment accounting, and persistent discharge/momentum maps in the erosion experiment. Compare it to the current solver on identical starting terrain. Check flow direction, stability, grid-alignment artifacts and sediment balance before judging its channels visually.

A tile border needs more than matching noise. Regional outlets and incoming flow must be agreed before local erosion. A finite halo can reduce local edge artifacts but cannot determine an unknown distant watershed. Cache solved regions and filter them into coarse LODs; do not rerun independent random erosion whenever a tile enters view. Generator-version changes must preserve already edited places or explicitly migrate them.

Fields and riverbanks: reserve broad, low-gradient floodplains and terraces; use deposition and moisture to distinguish exposed gravel, damp silt, meadow and cultivated land. Agricultural fields need parcels, access paths, hedgerows or ditches, crop growth and disturbance masks. They should not be identical dense grass painted onto every gentle slope. This is the proposed land-use design, not a claim that the cited erosion algorithm creates farms.

4. Smooth roads begin with route planning

Galin and colleagues explicitly show the limitations of eight-direction road search. Their method expands directional freedom, incorporates orientation/curvature, fits piecewise clothoid curves and treats construction operations such as bridges and terrain adaptation. This explains why applying a spline to an inadequate grid path cannot solve the whole road problem. [Galin, Peytavie, Maréchal and Guérin, Procedural Generation of Roads, Computer Graphics Forum, 2010.](#)

Recommended road hierarchy: inter-settlement routes connect destinations through suitable passes and crossings; local streets connect districts and gates; lanes provide actual frontage access to parcels and doors. Give these road classes different widths, allowable grades, turning behavior and construction costs. A ruined trail and a wagon route should not solve the same optimization problem.

Route with a heading-aware state or an equivalent curvature-aware solver. Sample longer directional candidates and integrate slope, water and obstacle costs along the entire segment. Encourage sensible junctions and reuse of established corridors. Select bridge crossings as spans with bank access and adequate clearance, rather than turning every wet path fragment into a separate bridge.

Fit a curvature-controlled curve inside the accepted corridor, then resample and revalidate its grade, turning radius, obstacles and water spans. Solve the vertical road profile and earthworks. The resulting single authoritative spline must drive the road surface, shoulder blend, terrain grading, vegetation clearance, bridge alignment and navigation. Remove the independent second smoothing pass.

Settlement layout should evolve with its roads. Emilien and colleagues use interest maps, road/building feedback, parcels and slope adaptation to generate villages. This provides a stronger starting point than dropping a house beside whichever road happens to be nearest. Their examples concern small European villages; use the spatial method while supplying appropriate architectural rules for other settings. [Emilien et al., Procedural Generation of Villages on Arbitrary Terrains, The Visual Computer, 2012.](#)

Acceptance: every required destination is connected; road and bridge surfaces meet without steps; no unexplained zigzags, double paths or stranded bridge fragments; buildings have usable frontage; curves remain smooth from eye level and aerial view. Numeric grade and turning thresholds must be authored per road class and tested against the game's movement rules.

5. Trees, plants and forests with recognizable character

Space colonization supplies controllable branching, but branching and foliage appearance are separate problems. Self-organizing tree research adds competition for space/light and developmental behavior. Use this evidence to enrich crown shapes, branch hierarchy and age, not to increase random branch counts indefinitely. [Runions, Lane and Prusinkiewicz, Modeling Trees with a Space Colonization Algorithm, 2007.](#) [Palubicki et al., Self-organizing Tree Models for Image Synthesis, ACM TOG, 2009.](#)

Plant recipes should describe branching habit, crown envelope, internode lengths, leaf arrangement and shape, bark scale, developmental stage, damage and wind stiffness. Use appropriate developmental grammars for herbs, ferns, reeds and flowers rather than forcing every plant through a tree generator. [Prusinkiewicz and Lindenmayer, The Algorithmic Beauty of Plants, Chapter 3, 1990.](#)

Replace the current large ellipsoid crowns with branchlets and leaf/needle arrangements close to the camera. At middle distance, use simplified foliage clusters or well-fitted cards. At longer distance, test multi-view tree proxies, then canopy-height/coverage representations. Preserve crown volume, gaps and color through transitions. Foliage LOD must address silhouette and shading as well as triangle count. [David Whatley, Toward Photorealism in Virtual Botany, GPU Gems 2, 2005.](#)

A forest needs communities: mature canopy, saplings, understory, litter, deadwood, clearings and disturbed edges. Deussen and colleagues separate ecosystem distribution from environment-dependent plant form and reuse similar plants or organs through instancing. Their scene-complexity results are not real-time frame-rate evidence. [Deussen et al., Realistic Modeling and Rendering of Plant Ecosystems, SIGGRAPH, 1998.](#)

Proposed implementation: create species families with correlated age/size variation; sample canopy light, moisture, soil, drainage and disturbance; distribute patches with exclusion and association rules; grow or choose prototypes appropriate to each patch. Use stable per-cell seeds, shared prototypes and compact instance records. Promote a plant to an individual persistent entity only when editing or gameplay requires it.

For foliage lighting, implement a controlled thin-leaf backlighting response, thickness variation and coherent normals, then verify both sides and shadow silhouettes. Wind should share a world field while trunk, branch and leaf responses differ; phases prevent every tree moving in lockstep. [Renaldas Zioma, GPU-Generated Procedural Wind Animations for Trees, GPU Gems 3, 2007.](#)

6. Dense grass and continuous biome transitions

Grass needs geometry that reads as blades at eye level, changes in height and density, root shading, wind, and believable coverage as LOD reduces its population. AMD demonstrates curved Bézier blades and distance reduction that compensates for lost coverage. Its implementation uses mesh shaders; the representation ideas can be adapted to instancing/compute, but its API and results are not a PlayCanvas performance guarantee. [Faber, Kuth, Meyer and Oberberger, Procedural Grass Rendering, AMD GPUOpen, 20 March 2024.](#)

Recommended grass hierarchy: near-camera blade clusters with bent profiles and species variation; fewer, coverage-preserving clusters in the middle range; then a terrain material carrying matching color, roughness and broad wind modulation. Crop fields additionally need row/patch structure and growth stages. Avoid full-detail grass beneath closed canopy, buildings, roads, deep water and distant terrain.

Maintain continuous environmental fields: temperature, seasonal precipitation, elevation, slope/aspect, drainage, soil, canopy light and disturbance. Derive overlapping plant-community weights from them. Use those same weights for terrain surfaces and vegetation. A forest-to-meadow transition should change canopy openness, understory and ground cover over space; changing the material at a chunk boundary is insufficient. This environmental field design is our synthesis, not a validated climate simulation.

The model may introduce fantasy communities—bioluminescent fungal forests, mineral meadows, floating gardens—through extensible plant and material rules. Their internal relationships should remain consistent even when they depart from Earth biology. The implementation should encode relationships and operators rather than a closed catalog of named biomes.

Millions of assets: distinguish the population implied by the world, the records resident in memory, the instances submitted for drawing, and the primitives actually shaded. Hardware instancing shares geometry, but it does not eliminate vertex processing, alpha overdraw, shadows, bandwidth or simulation work. PlayCanvas supports hardware instancing and WebGPU indirect drawing; the latter can consume compute-generated visibility and draw counts. A WebGL fallback needs its own bounded CPU/batch culling path. [PlayCanvas, Hardware Instancing](#). [PlayCanvas, Indirect Drawing](#). Documentation accessed September 2026.

One million 4×4 float matrices alone occupy 64 MB (about 61 MiB), before geometry, materials and metadata. This is arithmetic, not a benchmark. Compact transforms, cell-local coordinates and seed-based regeneration reduce storage; hierarchical culling and representation changes reduce rendering work. Give shadows and wind separate LOD budgets. Test foliage overdraw before assuming draw-call reduction solved the bottleneck.

7. Settlements and buildings that can actually be entered

CGA architecture uses hierarchical massing, façade subdivision and detail rules to produce building shells. Interior layout is a separate constrained problem: Merrell and colleagues generate an architectural program and optimize floor plans before creating the building. Our compiler needs both, linked by a common envelope and openings. [Müller et al., Procedural Modeling of Buildings, SIGGRAPH, 2006.](#) [Merrell, Schkufza and Koltun, Computer-Generated Residential Building Layouts, SIGGRAPH Asia, 2010.](#)

Recommended generation sequence: settlement purpose and terrain suitability; district and street graph; plots and frontage; building program; room/portal/stair graph; structural bays and envelope; façade/roof; semantic materials; functional furnishing and story sockets. Validate door-to-street access, room reachability, stair clearance, support on slopes and parcel overlap before dressing the scene.

For “an ancient Chinese village,” curved roofs alone are inadequate. Architectural identity includes plot and courtyard organization, structural proportions, enclosure, gates, roof construction, materials and region/period. The Hutong grammar study derives a hierarchy of neighborhood, plot, courtyard and rooms from historical Beijing examples. It is evidence for context-specific rules, not a template for all Chinese settlements. [Wang, Crompton and Agkathidis, The Hutong Neighbourhood Grammar, Frontiers of Architectural Research, 2023.](#)

References should influence proportions, repetition, construction and layout, as well as color. Image-based façade research demonstrates extracting hierarchical procedural structure from photographs; a single façade still cannot establish its unseen interior or complete building. [Müller et al., Image-Based Procedural Modeling of Facades, SIGGRAPH, 2007.](#)

The local model should compose and extend a construction vocabulary: footprints, extrusions, splits, repeated bays, roof profiles, stairs, wall openings, support structures and attachment rules. Store this compact hierarchy and compile several detail levels from it. Five global material roles should become semantic slots such as stone footing, plaster infill, structural timber, painted trim, roof tile and metal fittings.

At travel scale, preserve settlement extent and dominant roads, roofs, towers and walls. At local scale, refine those same buildings. Persist location, building, room and interaction IDs; returning must preserve opened doors, damage and DM-created content. Roof cutaway is useful presentation, but it does not substitute for traversable interiors or a room graph.

8. Materials, references and weathered surfaces

A material should produce related albedo, roughness, height, normal and other required channels from shared structural fields. Image brightness is not generally surface height: a dark paint stripe, shadow and recessed groove can look similar in one image. Treating all three as relief produces false bumps. Our current image-to-normal heuristic therefore cannot establish a complete physically coherent restyle.

Procedural material graphs offer an editable alternative. MATch demonstrates fitting graph parameters to reference appearance. That research supports the approach, but DiffMat's published license restricts it to noncommercial research and its implementation still describes practical Adobe-tool dependencies. Do not make it the default runtime dependency. [Shi et al., MATch, SIGGRAPH Asia, 2020](#). [DiffMat implementation and license](#), checked September 2026.

Material Maker is a candidate for studying or selectively porting procedural nodes: its repository license is MIT, subject to checking individual files and dependencies. It is Godot-based, so integration needs a controlled baking path or explicit node ports, not an assumption of direct PlayCanvas compatibility. [Material Maker, Rodolphe Suescun and contributors, license](#), checked September 2026.

Recommended local restyling: analyze user references into a style description; match semantic surfaces to material families; fit or generate shared structural parameters; bake coherent maps locally; test tiling, scale, normal orientation and lighting; publish the accepted material set atomically. Reuse maps and variations across instances. Material generation should be cancellable background work with app-owned GPU scheduling, not a per-frame operation.

Use macro color/roughness variation, physical texture scale, directional mapping for wood and roof tiles, triplanar mapping where appropriate, and anti-repetition sampling for natural surfaces. Randomized tiling with histogram-preserving blending is a researched way to reduce repetition without simply washing out contrast. [Brent Burley, On Histogram-Preserving Blending for Randomized Texture Tiling, JCGT, 2019](#).

Weather should modify causal masks and material response: exposed porous surfaces absorb water; sheltered interiors remain dry; puddles collect in depressions; snow responds to temperature, slope, exposure and shelter. Wetness changes are material-dependent, not a uniform gloss slider. [Sébastien Lagarde, Water Drop 3a: Physically Based Wet Surfaces, 19 March 2013](#).

For a strong offline baseline, curate legally redistributable material families and cache them with the app. Poly Haven states its assets are CC0; live website/API use has separate terms. This is an asset-library option, not a requirement for online scene generation. [Poly Haven asset license](#), checked September 2026.

9. Sky, weather and lighting should tell the same story

The sun that illuminates clouds must also illuminate terrain, foliage, roofs and water. Sky color, atmospheric attenuation, cloud shadows and exposure need one shared state. Distant mountains require atmospheric depth cues, but haze should not conceal missing geometry or make the entire world uniformly gray.

Hillaire's atmosphere reference accompanies the 2020 sky/atmosphere technique, uses lookup tables, and includes comparison modes. Its public implementation carries an MIT license; it is a candidate for a deliberate shader adaptation rather than a drop-in engine feature. [Sébastien Hillaire, A Scalable and Production Ready Sky and Atmosphere Rendering Technique, EGSR, 2020; reference implementation, license.](#)

Guerrilla's Nubis work is a practical source for cloud modeling and rendering. Its later superstorm presentation discusses weather coverage/type maps, optimized ray marching and temporal upscaling. These establish useful design directions; they do not demonstrate the cost of an equivalent WebView/WebGPU port. [Guerrilla, Nubis in a Nutshell, 2017. Guerrilla, The Real-Time Volumetric Superstorms of Horizon Forbidden West, GDC, 2022.](#)

Ubisoft's Far Cry 6 write-up provides a complementary example: regional weather controls, shelter-aware wetness, puddles and several atmospheric/rendering systems had to be tuned together. This supports treating weather as shared state rather than independent effects. [Ubisoft, Simulating Tropical Weather in Far Cry 6, accessed September 2026.](#)

Recommended weather state: world time, sun direction, temperature, wind vector/gusts, cloud coverage/type, precipitation, ground moisture and snow accumulation. Evaluate broad fields on a slow simulation cadence; update visible shading cheaply. All clients should receive the same weather state/seed. Keep permanent erosion separate from transient rainfall unless gameplay intentionally edits terrain.

Implement clear daylight first with calibrated surface scale, exposure and contact shadows. Add coherent sunset and overcast profiles; then wetness, snow and cloud shadows. Integrate aerial perspective with the same atmosphere used by the sky. Use a bounded cloud pass with depth-aware composition and test low-angle terrain silhouettes, temporal trails and transparency interactions. Avoid using bloom or depth of field to disguise shape or material problems.

Visual checks: rotate the sun and confirm every subsystem responds; follow the horizon while moving across LOD boundaries; enter a building during rain; compare wet wood, stone and polished metal; verify snow remains off warm sheltered surfaces; compare scene appearance at eye level and high travel-camera altitude. The current fixed snow threshold and cloud sun vector should be corrected before further aesthetic tuning.

10. Reveal the world through outward-moving volumetric fog

Required experience: the first visible area is one continuous patch of completed world. A volumetric fog front curls and rolls outward as more of the scene is revealed. Players should not see square tiles, empty edges, missing vegetation or separate construction stages pop into view. Internal streaming tiles can remain small and efficient; they must not define the visible presentation.

Use a world-space reveal field with a seeded, softly irregular boundary. Advect its detail with the same wind used by weather and foliage. Give the moving front depth, height variation, sunlight scattering and turbulent wisps; reduce density behind it as the completed terrain appears. This requires a spatially varying volume, not just a global distance-fog slider or a flat transparent circle. Unified volumetric rendering provides the rendering basis; the readiness-driven reveal is our proposed application design. [Sébastien Hillaire, Physically-Based and Unified Volumetric Rendering in Frostbite, SIGGRAPH course, 2015.](#)

Readiness controls the animation. A tile is eligible only when its accepted terrain, water, base materials, required vegetation/structures and GPU resources are ready. Promote it atomically under opaque fog. Advance a connected reveal region from the starting area; a stalled tile holds the front locally, so fog moves organically around completed areas without uncovering holes. Keep parent/coarse geometry available while finer children load. Cap reveal speed and maintain a hidden ready margin ahead of the visible front.

The current world worker returns a completed region in one message. It needs incremental, versioned readiness messages and transferable data, cancellation, and a bounded main-thread upload queue to support this honestly. Distinguish planned, generated, uploaded and visible states. Reporting generation progress alone is not enough to know whether the next area can be shown.

Camera behavior: normal movement prefetches beyond the fog boundary; teleports start with a covered destination and a ready coarse world before revealing detail. At high altitude, test that tall peaks and towers cannot protrude through the cover before they are ready. At eye level, test fog/ground contact and hidden tile joins. Reference-driven restyles should swap coherent material sets behind a controlled transition rather than displaying mismatched channels sequentially.

Gameplay separation: generation readiness, weather fog and player exploration visibility are independent state. The reveal effect may clear because rendering is ready, but it must not expose undiscovered interiors, enemies or DM secrets. A failed/cancelled generation retains the last valid visible area and a covered frontier; do not erase completed terrain or open holes. Any reduced-motion mode should preserve the soft reveal while reducing swirling motion.

Start with a bounded, depth-aware ray-marched volume at reduced resolution and verify upsampling/temporal behavior; consider a shared froxel volume if multiple local fog/weather effects justify it. This is a benchmark candidate, not a claim about current performance. Acceptance includes slow generation, out-of-order completion, sharp camera turns, teleports, abort/retry and simultaneous weather. Record video: still images cannot establish that chunk popping has disappeared.

11. Build order and evidence required before release

This is the recommended implementation order. The steps preserve the original local-AI and two-level gameplay requirements while replacing the finite-world assumptions.

Priority	Concrete work	Primary workspace targets
1	Capture stable baselines; isolate memory growth; record CPU/GPU costs and allocation lifetime	SceneViewport, worldChunks, worldGpuCompute
2	Shared metric world coordinates, tile addresses, rebasing, bounded cache/jobs, clipmap horizon	worldGeography, travelWorld, worldHorizon, persistence
3	Regional ridges/outlets, continuous erosion experiment, shared material/biome fields	worldProcedural, worldForge
4	One curvature/grade-aware road authority, bridges, parcels and frontage	worldProcedural, worldForge
5	Botanical recipes, realistic foliage, plant communities, instanced LOD and shadow budgets	worldArchitecture, worldForge, SceneViewport
6	Semantic architecture, room/portal graph, reference-driven construction/materials	sceneGrammar, sceneDirector, WorldJourneyPanel
7	Coherent atmosphere, weather, exposure and surface response	lightingEngine, volumetricClouds, tabletopShaders
8	Incremental readiness, atomic tile promotion and a contiguous outward fog reveal	worldForgeWorker, worldForgeWorkerClient, SceneViewport, new reveal controller/pass

Prove one connected journey first. Build a reference region with a mountain pass, river, meadow, dense forest and a settlement. Enter a building, interact with a DM-created point, leave, travel away and return. Include a culturally distinct second settlement to test whether the grammar is expressive rather than merely recolored. The region is a quality test fixture, not the boundary of the product world.

Use fixed camera paths at eye level, rooftop height and travel altitude, plus a grazing horizon view. Compare the same seed in dry daylight, overcast/rain and a cold/snow condition. Judge landform silhouette, plant silhouette, density layering, road plausibility, architectural identity, material scale, atmosphere and temporal stability separately. Review against user-provided references, not only against the previous screenshot. Prototype the reveal/readiness protocol with the tile scheduler in priority 2; finish its volumetric presentation after the shared atmosphere is established.

Performance gates are proposed targets: initially test 1080p with 16.7 ms and 33.3 ms frame budgets for separate quality tiers. Measure median/p95/p99 frame times, stutters, resident CPU/GPU memory, draw calls, visible instances, shadow cost, overdraw and tile generation/upload time. Record hardware, driver, WebView/backend, resolution and quality. Repeat traversal and teleports long enough to detect cache growth; test with the app-owned language model resident and with queued art generation. No source reviewed proves these targets are met.

12. Evidence limits and release boundaries

A million-instance test must state what the instances are, how many are visible, how many are drawn, their geometry and materials, and whether shadows are enabled. An empty instancing demo or a count of implied offscreen plants cannot certify a dense forest scene.

Remaining uncertainty: coherent infinite hydrology and edit propagation require an application-specific prototype; canopy proxies need low-angle validation; local reference-to-grammar reliability is unmeasured; actual native memory growth remains unresolved. The reviewed evidence supports the direction, but cannot guarantee arbitrary prompts produce excellent scenes. The current working changes have not been packaged as a new visually validated installer.

Research stopped after coverage of the material decision families, primary-source checks of major techniques and runtime/license constraints, and reconciliation with the source audit. Some large or moved PDFs were available only through indexed author/institution text or the independent research lane; no inaccessible implementation details are presented as verified. Further broad searching is less useful now than the connected visual/performance experiment above.